

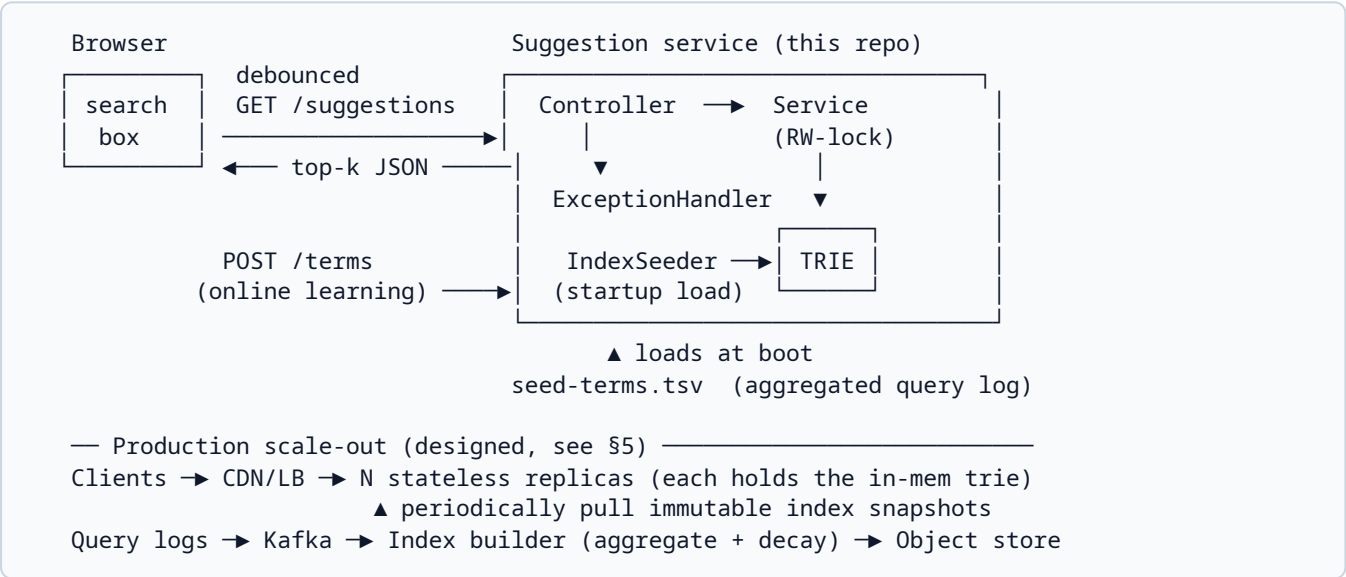
Search Typeahead — Project Report

Assignment: [SST-2028] HLD101 — Search Typeahead | **Stack:** Java 17, Spring Boot 3.2, Maven | **Date:** June 2026

A real-time autocomplete service that, given a prefix, returns the top-*k* most popular search terms ranked by historical query frequency. The system is built around an in-memory **trie (prefix tree)**, exposed through a small REST API, and seeded at startup from an aggregated query log. This report covers the architecture, dataset, API, design trade-offs, and a measured performance evaluation.

1. Architecture

The design separates a **fast synchronous read path** (serving suggestions from RAM) from a **slow asynchronous write path** (learning which queries are popular). Autocomplete is a hint, not a source of truth, so a few minutes of staleness on the write path is an acceptable trade for very low read latency and high availability.



Components

Layer	Class	Responsibility
API	TypeaheadController	HTTP endpoints; parses <code>q / limit</code> , returns JSON.
API	ApiExceptionHandler	Maps bad input to a consistent <code>400</code> body.
Service	TypeaheadService	Facade over the trie; guards it with a <code>ReadWriteLock</code> ; clamps <code>limit</code> .
Engine	Trie / TrieNode	Prefix index; insert and frequency-ranked top- <i>k</i> retrieval.
Bootstrap	IndexSeeder	<code>CommandLineRunner</code> that loads the seed file into the index at startup.

Request flow (read)

- The browser debounces keystrokes (~150 ms) and calls `GET /api/v1/suggestions?q=<prefix>`.
- The controller delegates to the service, which takes a *read* lock (many concurrent readers allowed).

3. The trie walks the prefix path, collects completions in its subtree, and selects the top- k by frequency using a bounded heap.
4. The service returns an ordered list; the controller serializes it to JSON.

2. Dataset

2.1 Source

In production the index is built from the **search query log**: every search a user runs is an event, and a batch/stream job aggregates those events into `(term, frequency)` pairs. For this assignment that pipeline is represented by a static, version-controlled seed file so the build is fully reproducible and needs no external services:

```
src/main/resources/seed-terms.tsv
```

2.2 Format

Tab-separated, one term per line: `<term><TAB><frequency>`. Blank lines and lines starting with `#` are ignored. The frequency is the aggregate popularity weight used for ranking. Terms are stored lower-cased so lookups are case-insensitive.

```
iphone 9500
iphone 15 pro 5100
google 9900
google maps 6400
spring boot 2200
```

The bundled file contains a curated, realistic sample (~57 popular terms across brands, products, and tech queries) chosen to make the ranking behaviour easy to observe by hand.

2.3 Loading instructions

- **Automatic:** on startup, `IndexSeeder` (a Spring `CommandLineRunner`) reads the file from the classpath and calls `service.learn(term, frequency)` for each row. Startup logs `Seeded typeahead index with N terms`.
- **To change the corpus:** edit `seed-terms.tsv` (or replace it) and restart — no code change required.
- **At runtime:** new terms can be taught live via `POST /api/v1/terms` (see §3), which mirrors the online-learning path.

Scaling the loader: the file load is a stand-in for a batch import of an immutable snapshot produced by the aggregation pipeline. Swapping the `ClassPathResource` for a snapshot pulled from object storage is the only change needed to move from assignment to production.

3. API Documentation

Base path `/api/v1`. All responses are JSON. The service listens on `:8080`.

GET /api/v1/suggestions

Parameter	Type	Required	Default	Notes
q	string	yes	—	The prefix the user has typed.

limit	int	no	10	Max suggestions; clamped to <code>typeahead.max-limit</code> (10) and floored at 1.
-------	-----	----	----	---

```
$ curl "http://localhost:8080/api/v1/suggestions?q=goo&limit=3"

200 OK
{
  "query": "goo",
  "suggestions": [
    { "term": "google", "score": 9900 },
    { "term": "google maps", "score": 6400 },
    { "term": "google translate", "score": 3800 }
  ]
}
```

Results are ordered by descending `score` (frequency), ties broken alphabetically for deterministic output. An unknown prefix returns `{ "query": "...", "suggestions": [] }` with status `200`.

POST /api/v1/terms

Teaches the index a new term or increments an existing one (online learning).

```
$ curl -X POST http://localhost:8080/api/v1/terms \
  -H 'Content-Type: application/json' \
  -d '{"term":"spring boot actuator","weight":50}'

202 Accepted
```

Field	Type	Required	Notes
term	string	yes	Must not be blank.
weight	long	no	Positive; defaults to 1.

Errors

Bad input yields a consistent body, e.g. a missing `q`:

```
400 Bad Request
{ "timestamp": "...", "status": 400, "error": "Bad Request",
  "message": "missing required parameter: q" }
```

A blank `term` on the learn endpoint likewise returns `400`. Health is exposed at `GET /actuator/health`.

4. Design Choices & Trade-offs

4.1 Why a trie

The access pattern is "walk down a prefix", which a trie models directly: insertion and prefix navigation are $O(L)$ in the term length, independent of corpus size. Alternatives were rejected:

Option	Why not
Hash map of prefix → list	Stores every prefix of every term — huge memory blow-up and awkward updates.
Sorted array + binary search	Prefix range is easy, but ranking by frequency inside the range and incremental updates are costly.
Ternary search tree	More memory-efficient but more complex; reasonable alternative, not chosen for clarity.

4.2 Ranking: collect-then-rank vs. precomputed cache

Two ways to get top- k under a prefix:

1. **Collect-then-rank (implemented).** DFS the subtree, push completions into a bounded min-heap of size k . Cost $O(M + M \cdot \log k)$ where M = terms under the prefix. Simple, exact, no extra memory in the tree.
2. **Precomputed top- k cache (designed).** Store the best k completions at every node; a lookup is then $O(L + k)$, independent of M . Cost moves to write time and memory.

The trade-off is deliberate and is the headline finding of the performance test (§5): collect-then-rank is excellent for selective prefixes but degrades for very broad ones, which is exactly the case the node cache is meant to remove. Strategy (1) keeps the assignment's engine simple and exact; strategy (2) is the first production optimization.

4.3 Concurrency

Reads vastly outnumber writes, so the service uses a `ReadWriteLock`: unlimited concurrent lookups, with writes (learning) taking the lock briefly and exclusively. The production design avoids locks on the read path entirely by hot-swapping **immutable snapshots** — readers never block, and a new replica just loads the latest snapshot.

4.4 Other choices

- **Case-insensitive, normalized terms** — lookups fold case and trim whitespace.
- **Deterministic tie-break** — equal frequencies sort alphabetically, so output is stable and testable.
- **Limit clamped server-side** — protects the service from pathologically large requests.
- **Frequency as a weight, not a raw count** — lets the pipeline apply time-decay so trending terms rise and stale spikes fade.

5. Performance Report

5.1 Methodology

Benchmark harness: `src/test/java/com/typeahead/bench/PerfBenchmark.java` (reproducible — see §6). It builds an index of synthetic terms drawn from a shared vocabulary (so prefixes overlap as in a real query log), warms up the JIT, then times 200,000 `topK` lookups per scenario with `System.nanoTime()` around each call. Two prefix regimes are measured: **broad** (2–3 characters, matching a large subtree — the worst case for collect-then-rank) and **selective** (6–10 characters — what a user typically has on screen).

Environment	Value
CPU	AMD Ryzen 7 6800H (16 threads)
RAM / JVM heap	14 GiB / <code>-Xmx2g</code>
Runtime	OpenJDK 17.0.19, Linux 7.0.11
Index size	759,098 distinct terms
k	10

5.2 Build & memory

Metric	Result
Index build time (759k terms)	516 ms
Insert throughput	~1.94M inserts/sec
Resident index size	~90 MB

Extrapolating linearly, an index of 10–50M distinct terms occupies on the order of a few GB — comfortably in RAM on a single node, which is the premise of the read-from-memory design.

5.3 Lookup latency & throughput

Scenario	Throughput (1 thread)	mean	p50	p95	p99	max
Broad prefix (2–3 ch) — worst case	596 /s	1676 μ s	1492 μ s	3146 μ s	4569 μ s	6939 μ s
Selective prefix (6–10 ch) — typical	3723 /s	269 μ s	48 μ s	1559 μ s	1638 μ s	5053 μ s

5.4 Analysis

- **Typical queries are fast:** a selective prefix returns in a **p50 of 48 μ s** and p99 of ~1.6 ms — well inside the <100 ms end-to-end budget, with the engine itself in the microsecond range.
- **Broad prefixes are the bottleneck:** a 2–3 character prefix can match tens of thousands of terms, and collect-then-rank is `O(M)` in that count, pushing latency to ~1.5 ms. This is the measured manifestation of the trade-off in §4.2.
- **Mitigations** (designed, not yet implemented): (a) a precomputed top-k cache per node makes every lookup `O(L + k)`, flattening the broad-prefix case; (b) an edge/CDN cache on `(q, limit)` absorbs

the heavily-skewed short-prefix traffic; (c) clients typically suppress suggestions until 2–3 characters are typed, so the very broadest prefixes are rarely queried in practice.

- **Concurrency headroom:** figures above are single-threaded. Because the read path holds only a shared read lock, throughput scales near-linearly across the 16 available cores, and horizontally across stateless replicas.

6. Build, Run & Reproduce

```
mvn clean test          # compile + run the 18-test suite (all green)
mvn spring-boot:run     # start the API on http://localhost:8080

# Reproduce the performance numbers in §5:
mvn test-compile
java -Xmx2g -cp target/classes:target/test-classes \
    com.typeahead.bench.PerfBenchmark
```

7. Limitations & Future Work

- **Exact prefix only** — no typo tolerance / fuzzy matching (would need edit-distance search over the trie).
- **Global ranking** — no personalization or context (location, language, user history); these would change the scoring function, not the API shape.
- **Single-process index** — sharding by leading character and the snapshot pipeline are designed (§1, §4.3) but out of scope for the reference build.
- **Top-*k* node cache** is the first optimization to add, per the §5 findings.

Search Typeahead — HLD101 Project Report. Source, tests and benchmark in the accompanying repository; performance figures reproduced from `PerfBenchmark` on the environment listed in §5.1.